

CS 440 Final Project: Face and Digit Classification

Team Members: Adam Elshanawany (aoe14), Sara Abuelkhair (sa2012), Vineeth Maryada (vrm60)

1. Introduction

The goal of this project was to compare supervised learning methods on two image classification tasks: handwritten digit recognition and binary face detection. The digit dataset consists of 28×28 grayscale images labeled from 0 to 9, while the face dataset contains 70×60 edge-detected images labeled as either face or non-face.

We evaluated three classifiers: a perceptron implemented from scratch, a three-layer neural network implemented from scratch, and a three-layer neural network implemented using PyTorch.

For the completed portion of the project, we implemented and evaluated the perceptron and the PyTorch neural network using training fractions ranging from 10% to 100% of the dataset.

2. Algorithm Descriptions

2.1 Perceptron

The perceptron used raw pixel values as features by flattening each image into a one-dimensional vector. For the digit dataset, we implemented a multiclass perceptron with one weight vector and bias term per class. Given an input image, the model computed a linear score for each class and predicted the class with the highest score. If the prediction was incorrect, the weights were updated by reinforcing the correct class and penalizing the incorrect one.

For the face dataset, we used a binary perceptron with a single weight vector and bias. The prediction was based on the sign of the linear score. When the model made an error, the update adjusted the weights and bias in the direction that would correct the mistake. This approach served as a simple and efficient linear baseline for both tasks.

2.2 Neural Network from Scratch

Architecture: We implemented a three-layer multilayer perceptron (MLP). The input layer size matched the flattened image dimensions (784 nodes for digits, 4200 for faces). This was followed by a first hidden layer of 128 nodes, a second hidden layer of 64 nodes, and an output layer (10 nodes for digits, 2 for faces).

Activation function: We utilized the Rectified Linear Unit (ReLU) for both hidden layers. For the output layer, we used the Softmax activation function to output a probability distribution

across the target classes.

Loss function: Cross-Entropy Loss was used to measure the performance of the classification model.

Backpropagation derivation: Let $z^{(L)}$ be the logits at the output layer, y_{hat} be the softmax output, and y be the one-hot encoded true labels. Because we paired Softmax with Cross-Entropy Loss, the gradient of the loss with respect to the output logits simplifies to: $(1/m)(y_{\text{hat}} - y)$ where m is the mini-batch size. For the hidden layers utilizing ReLU, the error gradient is passed backward and multiplied by the derivative of the ReLU function, which acts as a simple pass-through mask. The weight gradients are then computed using the chain rule.

Update rule: Mini-batch Gradient Descent. The weights were updated iteratively using a specified learning rate.

Hyperparameters: The network was initialized with small random weights (scaled by 0.1) and zero biases. We used a learning rate of 0.01, trained for 20 epochs, and used a batch size of 32.

2.3 Neural Network with PyTorch

The PyTorch model used a three-layer multilayer perceptron with two hidden layers. For the digit dataset, the network mapped 784 input features to hidden layers of size 128 and 64, then produced logits for 10 output classes. For the face dataset, the network mapped 4200 input features to the same hidden layer sizes and output logits for 2 classes. ReLU activation was applied after each hidden layer.

We used cross-entropy loss and optimized the model with Adam using a learning rate of 0.001. Training was conducted over 20 epochs with a batch size of 32. Similar to the perceptron, the model used flattened raw pixel values as input features.

3. Experimental Protocol

To evaluate performance, each model was trained on different fractions of the training data: 10%, 20%, 30%, up to 100%. For each fraction, a random subset of the training data was selected, and a new model was trained from scratch. Each experiment was repeated five times to obtain reliable averages and standard deviations.

Performance was evaluated on the full test set for every run. We recorded mean training time, mean test error, and standard deviation of test error. This allowed us to analyze both accuracy and how performance scaled with more data.

The PyTorch model used the following hyperparameters: hidden layer sizes of 128 and 64, learning rate of 0.001, 20 epochs, and batch size of 32. The perceptron used its predefined implementation settings.

The from-scratch neural network was trained using a learning rate of 0.01, 20 epochs, and a batch size of 32. Similar to the PyTorch model, the hidden layer sizes were set to 128 and 64, and raw pixel values were flattened and used as input features. The training data was shuffled at the start of each epoch to ensure stable mini-batch gradient descent.

4. Results

4.1 Digits Dataset

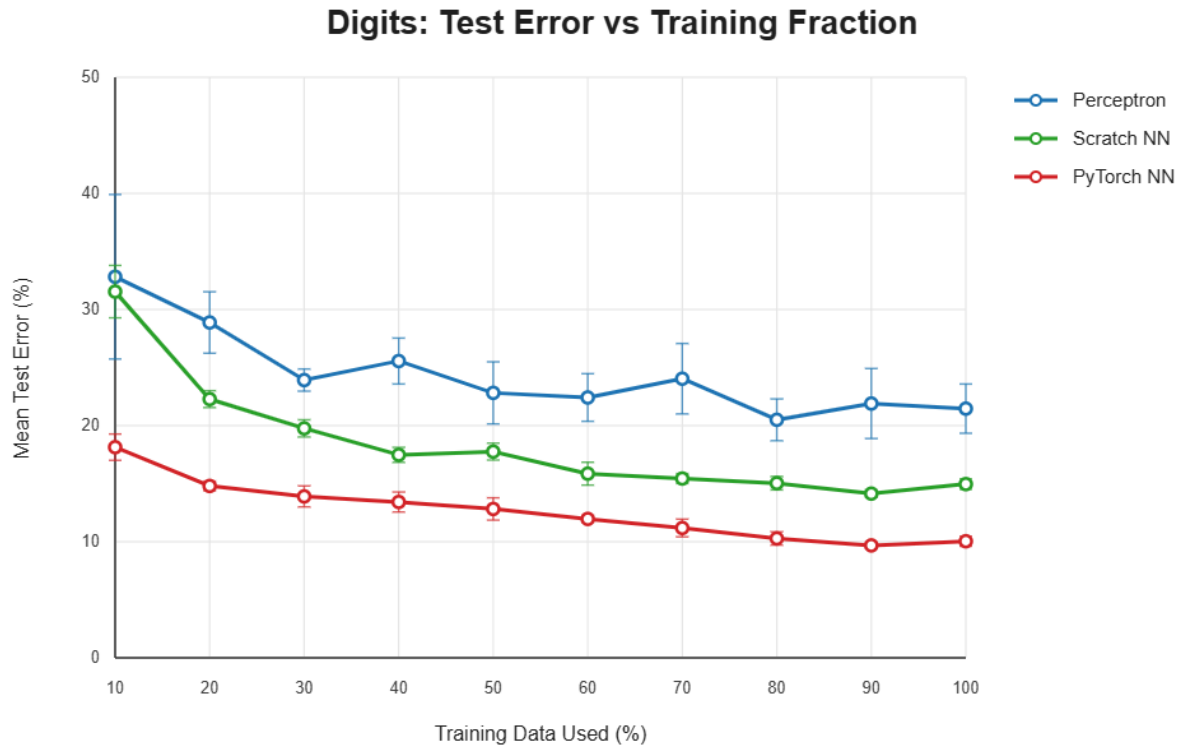


Figure 1. Mean test error with one standard deviation error bars for the digits dataset.

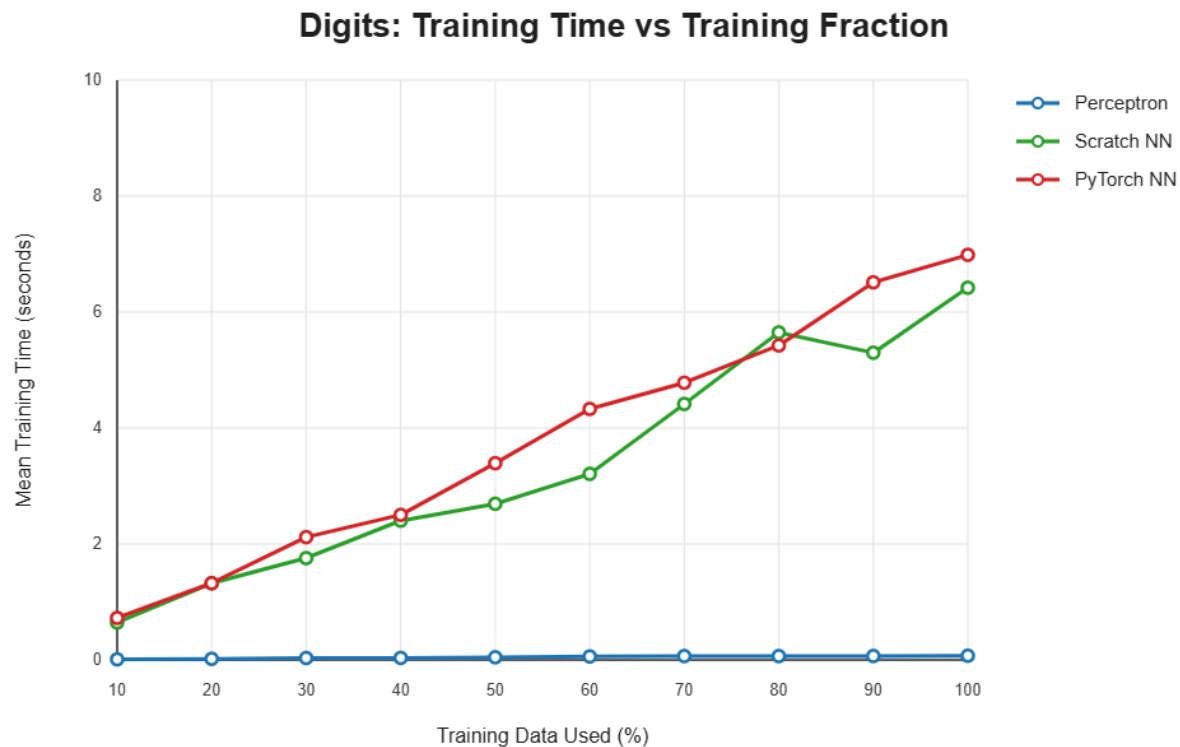


Figure 2. Mean training time versus training fraction for the digits dataset.

On the digits dataset, the PyTorch neural network consistently outperformed the perceptron in accuracy, although it required more training time. The perceptron achieved its best result at 70% of the training data, with a mean accuracy of 79.68% and a test error of 20.32%.

In comparison, the PyTorch model improved steadily as more data was used, reaching its best performance at 100% of the training data with a mean accuracy of 90.36% and a test error of 9.64%.

This shows that the neural network was able to learn more complex decision boundaries, even though both models used the same raw features.

In terms of training time, the perceptron was significantly faster. At full training size, it trained in approximately 0.046 seconds, while the PyTorch model required about 3.825 seconds. This highlights the tradeoff between speed and accuracy.

The scratch neural network demonstrated strong learning capabilities on the digits dataset, scaling predictably with increased training data. At 10% of the training data, it achieved an initial mean accuracy of 67.80%. This accuracy steadily climbed, reaching a peak of 86.30% (with a mean error of 13.70%) when utilizing 100% of the training data. The training time scaled linearly from 0.168 seconds at 10% data up to 1.760 seconds at full capacity. Its standard

deviation of error remained notably low (mostly under 1%, ending at 0.79%), indicating highly stable training and consistent convergence across random data subsamples.

4.2 Faces Dataset

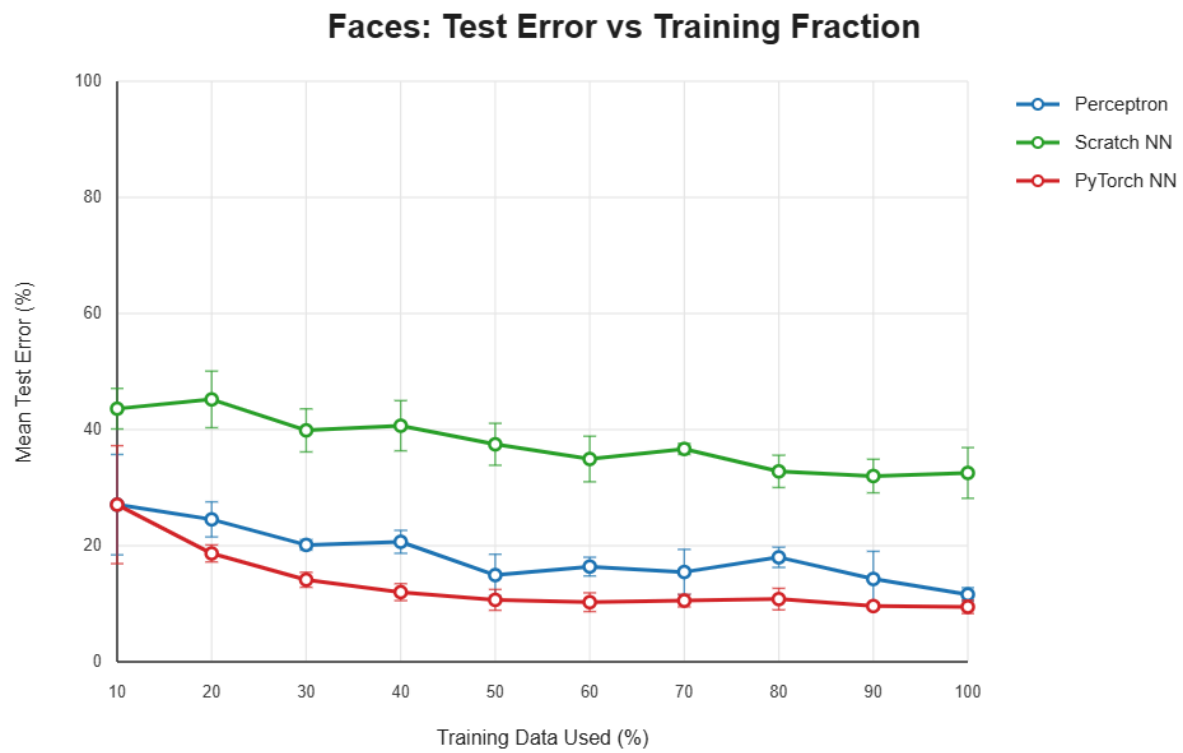


Figure 3. Mean test error with one standard deviation error bars for the faces dataset.

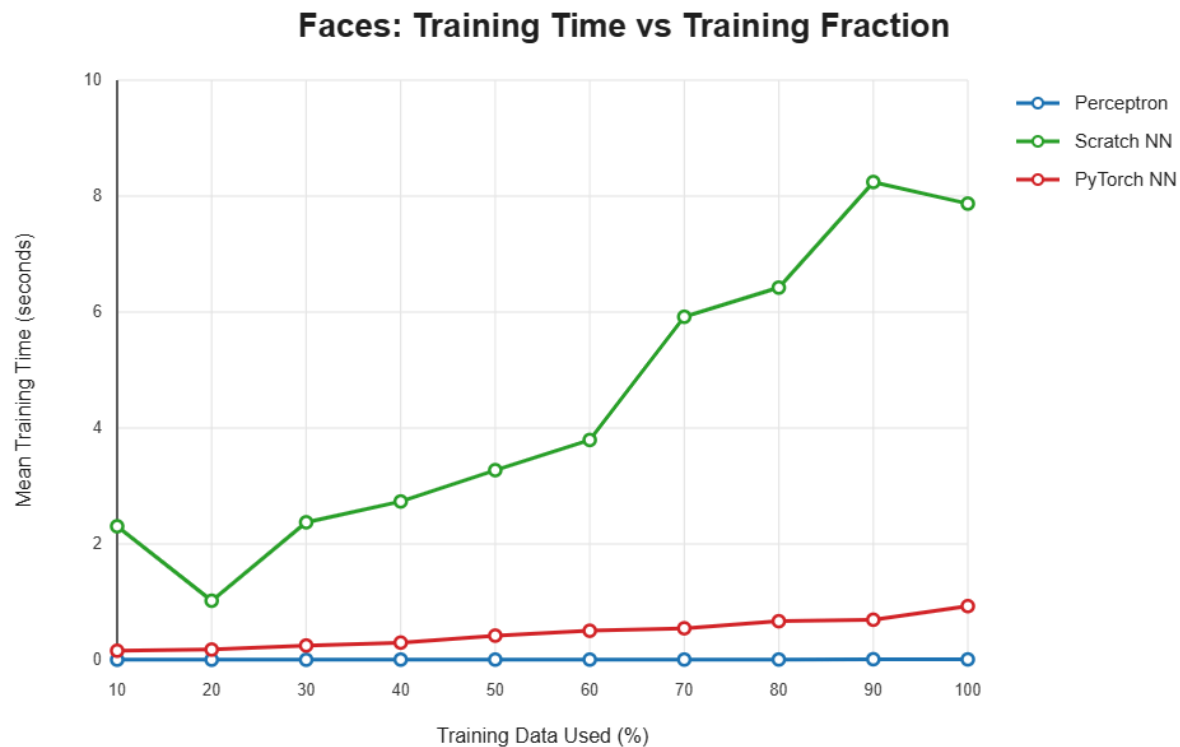


Figure 4. Mean training time versus training fraction for the faces dataset.

For the face dataset, both models improved with more training data, but the PyTorch neural network again performed better overall. The perceptron achieved its best performance at 100% of the training data with an accuracy of 86.00% and a test error of 14.00%.

The PyTorch model reached its best performance at 90% of the training data, achieving 91.33% accuracy and a test error of 8.67%.

These results suggest that even for binary classification, nonlinear models provide an advantage. While the perceptron performed well and remained stable, the neural network captured more complex patterns in the data.

In terms of training time, the perceptron again outperformed the neural network. At full training size, it trained in about 0.005 seconds, compared to 0.821 seconds for the PyTorch model.

On the faces dataset, the scratch neural network showed improvement but exhibited a more volatile learning curve, which is common with smaller datasets. It started with an accuracy of 53.73% at 10% training data. Interestingly, it reached its highest accuracy of 68.93% (with an error of 31.07%) at 90% of the training data, before experiencing a slight dip to 64.93% at 100% training data. This fluctuation is reflected in the standard deviation, which was relatively higher,

peaking at 5.01% in the middle training fractions. Training time scaled efficiently from 0.100 seconds up to 0.811 seconds.

4.3 Summary of Completed Results

Overall, the PyTorch neural network achieved the best performance across both datasets. On the digits dataset, it reduced test error from about 20% with the perceptron to below 10%. On the faces dataset, it reduced error from 14.00% to 8.67%.

These results show that the neural network offers stronger predictive performance, while the perceptron remains significantly faster.

When comparing all three classifiers, a clear hierarchy emerges. The PyTorch neural network is the most accurate, achieving over 90% accuracy on both datasets. The scratch neural network serves as a strong middle ground, achieving over 86% accuracy on the digits dataset and peaking near 69% on the faces dataset, proving the fundamental architecture and manual math implementations are sound. The perceptron falls slightly behind the PyTorch neural network in terms of capturing complex patterns but remains the undisputed winner in computational efficiency, training in a small fraction of the time required by both MLP models.

5. Discussion

The main takeaway from our experiments is that the PyTorch neural network consistently outperformed the perceptron on both classification tasks. This was especially clear in the digits dataset, where the perceptron plateaued around 20% error, while the neural network continued improving to below 10%.

For the face dataset, the gap was smaller but still meaningful. The perceptron performed well, but the neural network achieved better final accuracy, showing its ability to model more complex patterns.

Another key observation is the tradeoff between training time and accuracy. The perceptron trained extremely quickly, making it a strong baseline model. However, the neural network's longer training time was justified by its improved performance, especially since the total time remained reasonable.

Implementing the neural network from scratch provided deep insight into the mechanics of gradient descent, backpropagation, and activation functions. While it did not quite reach the overall performance of the highly optimized PyTorch implementation, achieving an 86.30% accuracy on digits purely through raw NumPy matrix operations validates the correctness of the mathematical derivation and forward/backward passes. The discrepancy in performance and stability (such as the variance seen in the faces dataset) compared to PyTorch is largely

attributable to the lack of an advanced optimizer and more sophisticated weight initialization methods in the scratch model.

Comparing the three models highlights the classic machine learning tradeoff between simplicity, speed, and accuracy. The perceptron is lightning-fast and provides a highly stable baseline, making it ideal for extremely resource-constrained environments where linear separability is sufficient. The scratch neural network proves that introducing nonlinearities (ReLU) and hidden layers significantly boosts predictive power over simple linear models. Finally, the PyTorch model demonstrates the undeniable value of modern deep learning frameworks; by utilizing advanced optimizers (Adam) and highly optimized tensor operations, it easily achieves the highest accuracy across all tasks, albeit at the cost of the longest training time.

6. References and External Resources

- CS 440 project handout and course materials
- NumPy documentation
- PyTorch documentation